



MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
MOBİL PROGRAMLAMA DERSİ FİNAL PROJESİ

Task Manager

Mobil Programlama Final Projesi

Mustafa Başkonus - 19120205003

Berke Karaca - 20120205306

Ders Sorumlusu

Dr. Öğr. Üyesi Muhammet Sinan BAŞARSLAN

Ocak, 2025

İstanbul Medeniyet Üniversitesi, İstanbul

İÇİNDEKİLER

	Sayfa No
İÇİNDEKİLER	i
TABLO LİSTESİ	ii
ŞEKİL LİSTESİ	ii
ÖZET	iv
SUMMARY	vi
1. GİRİŞ	1
2. LİTERATÜR TARAMA/İLGİLİ ÇALIŞMALAR	2
3. MATERYAL METOD	3
3.1 Flutter: Projede Kullanılan Yazılım Teknolojisi	3
3.2 İDE	4
3.3 SQFlite: Backend Yönetiminde Kullanılan Veritabanı	5
4. UYGULAMA	6
4.1 Önyüz Tasarım	6
4.2 Veritabanı	8
5.3 Uygulama Arayüzleri	9
5. SONUÇ VE TARTIŞMA	12
6. KAYNAKÇA	13

TABLO LİSTESİ

	<u>Sayfa No</u>
Tablo 2.1. Literatür Taraması	2

ŞEKİL LİSTESİ

	<u>Sayfa No</u>
Şekil 3.1. Flutter Platformları	4
Şekil 3.2. Flutter İDE	5
Şekil 4.1. Anasayfa ve Tüm görevler Sayfası Tasarımı	6
Şekil 4.2. Kategori ekran TasarımıŞekil	7
Şekil 4.3. Görev Ekle ekran Tasarımı	7
Şekil 4.4. DB Tablo tasarımı	8
Şekil 4.5. Anasayfa Ekranı	9
Şekil 4.6. Tüm Görevler Ekranı	10
Şekil 4.7. Bir Kategoriye Ait Görevler Ekranı	10
Şekil 4.8. Görev ekle Ekranı	11
Şekil 4.9. Görev güncelle Ekranı	11

ÖZET

Task Manager, günlük yaptığımız işlevlerin bir hatırlatıcısı olma görevini üstlenmektedir. Günlük rutinlerinizde hatırlamak isteyeceğiniz her başlığı kategori olarak ekleyebileceğiniz ve kendinize göre özelleştirebileceğiniz bir uygulama olmakla birlikte; uyku saati, ilaç içme saati ve ders saati uygulamanın başlangıç sürümünde manuel olarak hazırlanmış olarak gelmektedir. Uygulamanın özel ve bizce diğer uygulamalardan ayıran kısımlarından birisi de size bildirim olarak istediğiniz periyotlarda hatırlatma sunma özelliğidir. Canlı bir takip gerektirmeyen uygulama, sizin seçtiğiniz periyotlarda size bildirim olarak hatırlatma yapma özelliğine sahiptir. Genel olarak “Task” olarak belirlediğimiz görevleri hayatımıza entegre etmemizde bize yardımcı olur.

Anahtar Kelimeler: Task ,Flutter, Sqlflite.

1. GİRİŞ

Günümüzde mobil uygulamalar, bireylerin günlük yaşamlarını düzenlemelerine ve verimliliklerini artırmalarına yardımcı olan önemli araçlar haline gelmiştir. Özellikle görev yönetimi ve zaman planlaması konularında geliştirilen uygulamalar, kullanıcıların işlerini organize etmelerini kolaylaştırmaktadır. Bu bağlamda, Flutter kullanılarak geliştirilen ve TableCalendar kütüphanesi ile entegre edilen bu uygulama, kullanıcıların belirli kategoriler altında görev ekleyip yönetmelerine olanak tanımaktadır.

Bu raporda, geliştirilen uygulamanın temel özellikleri, teknik altyapısı ve kullanım senaryoları detaylı olarak ele alınacaktır. Uygulama, dört temel kategori üzerinden görevleri sınıflandırmayı mümkün kılmakta olup, kullanıcıların ders, ilaç takibi, uyku düzeni ve genel görevlerini takip etmelerini sağlamaktadır. Ayrıca, uygulamada görev ekleme, düzenleme ve silme gibi temel işlevler bulunmaktadır. Kullanıcı dostu arayüzü ve etkin veri yönetimi sayesinde, bu uygulama kişisel zaman yönetimi ihtiyacına yönelik pratik bir çözüm sunmaktadır.

Bu çalışma kapsamında, uygulamanın teknik detayları, kod yapısı ve işlevselliği analiz edilerek geliştirilen sistemin etkinliği değerlendirilecektir. Ayrıca, benzer çalışmalar incelenerek literatür taraması yapılacak ve elde edilen bulgular doğrultusunda uygulamanın geliştirilmesine yönelik öneriler sunulacaktır.

2. LİTERATÜR TARAMA/İLGİLİ ÇALIŞMALAR

Bu bölümde Google üzerinden tarama yapılarak uygulamamıza benzer ve ilham verici ayrıntılar taşıyan bazı makalelere yer verilmiştir. Tablo 2.1. de gördüğünüz gibi yerel ve global piyasada bu konuda uygulama denemeleri olmuştur.

Tablo 2.1. Literatür Taraması

MAKALE/TEZ ADI	MAKALE/TEZ YAZARI/YAZARLARI	YAYINCI	MAKALE/TEZ KONUSU
Flutter ile to-do list uygulaması yapmak [1]	Zahid Tekbaş	Zahid Tekbaş'ın kişisel blogu	Flutter ve Sqflite kullanarak bir yapılacaklar listesi uygulamasının nasıl geliştirileceği adım adım anlatılmaktadır
Developing a Professional Flutter Task Management Application [2]	Mohammad Sakib Mahmood	Mohammad Sakib Mahmood'un Medium blogu	Flutter kullanarak profesyonel bir görev yönetimi uygulamasının nasıl geliştirileceği adım adım anlatılmaktadır.

Tablo 2.1. de görüldüğü üzere Flutter ve Sqflite kullanarak yapılan diğer uygulamalar bu şekildedir.

3. MATERYAL METOD

Projemizde aşağıdaki materyaller ve metotlar kullanılmıştır.

- Proje yazılım dili olarak Flutter seçtik. Flutter seçmemizdeki en önemli neden bu dilin cross platform özelliği idi.
- Projeyi geliştirme ortamı olarak VSCode ve Androit Studio'yu birlikte kullanıldı.
- Verilerimizi kaydetmek için ilişkisel bir veri tabanı kullanmak istedik bu yüzden de SQLite veri tabanı kullanıldı.
- Projemizin UI tasarımlarını yaparken herhangi bir ortam kullanmadan kağıt üzerinde tasarlandı.
- Material Design uygulama bileşenleri için kullanıldı.
- Takvim için de table calendar (sürüm ^3.1.3) kullanıldı.[1]

3.1 Flutter: Projede Kullanılan Yazılım Teknolojisi

Bu projede, 2017 yılında Google tarafından geliştirilen **Flutter** yazılım teknolojisi kullanılmıştır. [2]

Peki neden mobil uygulama geliştirmede Flutter dilini seçtik. Hem yüksek performanslı hem de çapraz platformda çalışan modern ve kullanıcı dostu uygulamalar geliştirmek için. Flutter'ın en büyük avantajlarından biri, aynı kod tabanını kullanarak hem Android hem de iOS platformlarına uygulama geliştirme imkanı sağlamasıdır.

Flutter'ın Özellikleri:

- **Hızlı Geliştirme:** Hot Reload özelliği sayesinde, yapılan değişiklikler anında görülebilir. Bu da geliştiricilere daha verimli bir geliştirme süreci sunar.
- **Esnek ve Modern UI:** Flutter, özel widget yapıları sayesinde hem modern hem de kullanıcı dostu arayüzlerin kolayca tasarlanmasını sağlar.
- **Performans:** Flutter, Dart dilini kullanarak yüksek performanslı uygulamalar geliştirme olanağı sunar. Yerel (native) derleme özelliği sayesinde, uygulamalar kullanıcıya hızlı bir deneyim sunar.
- **Çapraz Platform Desteği:** Tek bir kod tabanıyla Android, iOS, web ve masaüstü platformları için uygulama geliştirmek mümkündür. Bu da zaman ve maliyet tasarrufu sağlar.

Neden Flutter?

Flutter, sadece mobil uygulama geliştirme sürecini kolaylaştırmakla kalmaz, aynı zamanda esnek yapısı ve güçlü performansı ile geliştiricilere modern çözümler sunar. Google'ın 2017 yılında düzenlediği etkinliklerde, Flutter'ın hem kullanıcı deneyimini hem de geliştirici deneyimini artırmak için tasarlandığını açıklaması, Flutter'ı hızlı bir şekilde popüler bir tercih haline getirmiştir. [2]

Flutter'ın çıkış amacı; modern arayüzlerle hızlı, performanslı ve çapraz platform uygulamaları geliştirilmesine olanak tanıyan bir yazılım geliştirme çerçevesi sunmaktır.

Şekil 3.1.'de Flutter dilinin hizmet verdiği platformlar görülmektedir.



Şekil 3.1 Flutter Platformları [7]

3.2 İDE

Android Studio: IDE

Proje geliştirilirken kullanılan geliştirme ortamı olarak **Android Studio** tercih edilmiştir. Çünkü Android Studio, özellikle Android uygulama geliştirme süreçleri için optimize edilmiş, Google tarafından desteklenen resmi bir entegre geliştirme ortamıdır (IDE). Android Studio, özellikleri sayesinde geliştiriciler için güçlü bir araçtır.[3]

Visual Studio Code: IDE

Projede kullanılan bir diğer geliştirme ortamı ise **Visual Studio Code** olmuştur. Visual Studio Code, Microsoft tarafından geliştirilen ve geniş bir kullanıcı kitlesine sahip hafif, hızlı ve özelleştirilebilir bir metin editörüdür. Çeşitli programlama dillerini ve teknolojileri desteklemesiyle bilinir. Flutter ve diğer çapraz platform projelerinde sıklıkla tercih edilmektedir. [4]

Şekil 3.2.'de projede kullanılan İDE logoları verilmiştir.



Şekil 3.2 Flutter İDE

3.3 SQFlite: Backend Yönetiminde Kullanılan Veritabanı

SQFlite, Flutter projelerinde lokal veri saklama ve yönetimi için kullanılan bir SQLite eklentisidir. SQL tabanlı sorgularla çalışan SQFlite, cihaz üzerinde hızlı ve güvenilir veri yönetimi sağlar.[5]

Temel Özellikleri:

- Çapraz platform desteği (Android ve iOS)
- Performanslı ve hafif yapı
- İnternet gerektirmeyen yerel veri saklama
- CRUD işlemleri için güçlü SQL desteği
- İlişkisel veri tabanı

Bu projede, kullanıcı verileri ve uygulama içeriği, SQFlite kullanılarak tablolar üzerinden güvenli ve esnek bir şekilde yönetilmektedir.[6]

4. UYGULAMA

Bu bölümde, projenin geliştirme sürecinde uygulanan tasarım, veritabanı yapısı ve arayüzler ele alınmıştır. Kullanıcı dostu bir deneyim sağlamak ve uygulamanın işlevselliğini artırmak amacıyla tüm süreçler detaylı bir şekilde planlanmış ve hayata geçirilmiştir.

4.1 Önyüz Tasarım

Projenin ön yüz tasarımı, kullanıcı deneyimini geliştirmek ve işlevselliği artırmak amacıyla öncelikle kağıt üzerinde tasarlanmıştır. Bu yöntem, tasarım sürecinde ekranların düzenini ve akışını görselleştirmeye olanak sağlamıştır.

Tasarlanan Ekranlar:

a) **Kategori Sayfası:**

Kullanıcıların kategorileri liste halinde görüntüleyebileceği ana ekrandır. Kategori kartları, kategori adı ve simgesini içerecek şekilde tasarlanmıştır.

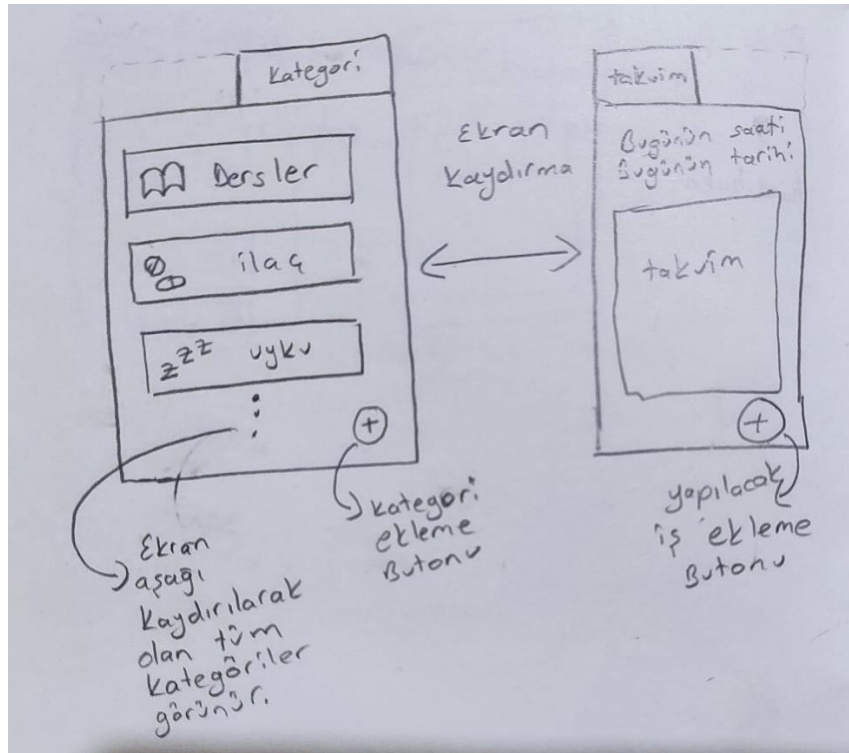
b) **Kategori Detay Sayfası:**

Bir kategori kartına tıklanıldığında açılan ekrandır. Bu sayfada, ilgili kategoriye ait görevlerin listesi gösterilir. Ayrıca kullanıcı, mevcut görevleri düzenleyebilir veya yeni görevler ekleyebilir.

c) **Görev Ekleme Sayfası:**

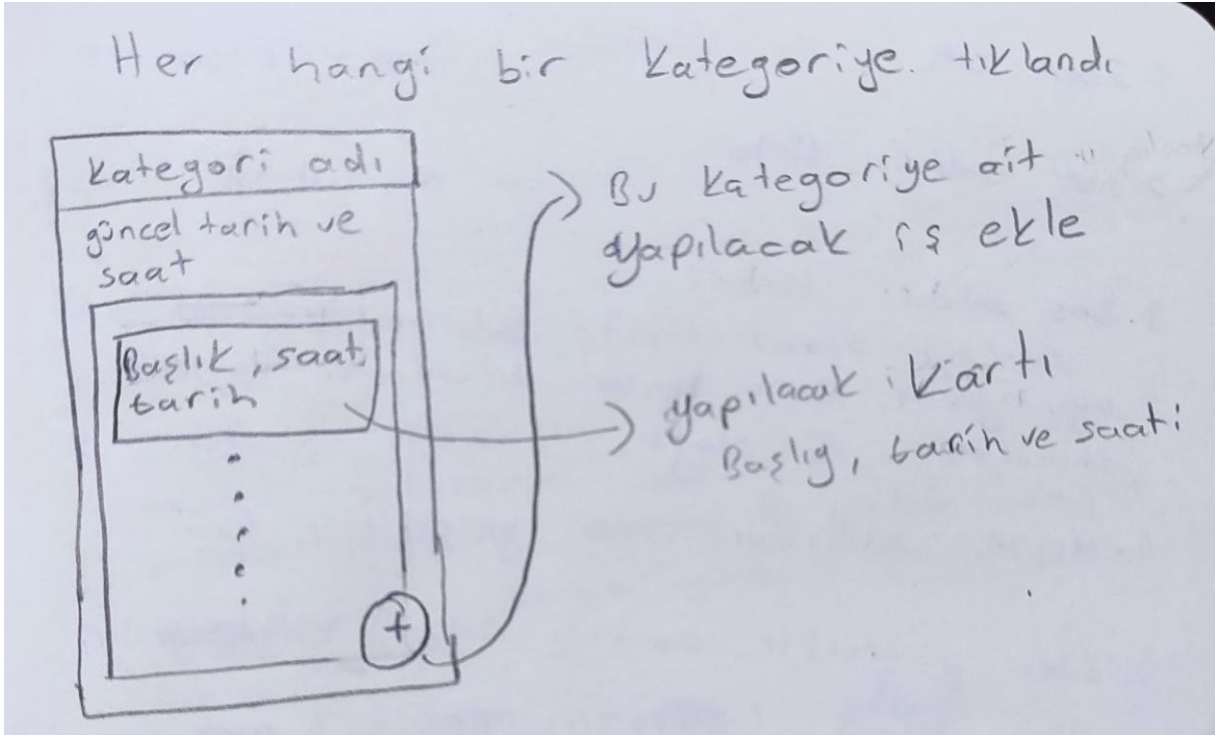
Kullanıcıların yeni görevler oluşturabileceği bir ekrandır. Tasarımda, görev başlığı, tarih seçimi ve kategori seçimi gibi alanlar yer almaktadır. Kullanıcı dostu bir arayüz sunmak için giriş alanları ve düğmeler sade ve anlaşılır bir şekilde düzenlenmiştir.

Şekil 4.1.'de projeye ait Anasayfa ve Tüm görevler Ekran Tasarımı verilmiştir.



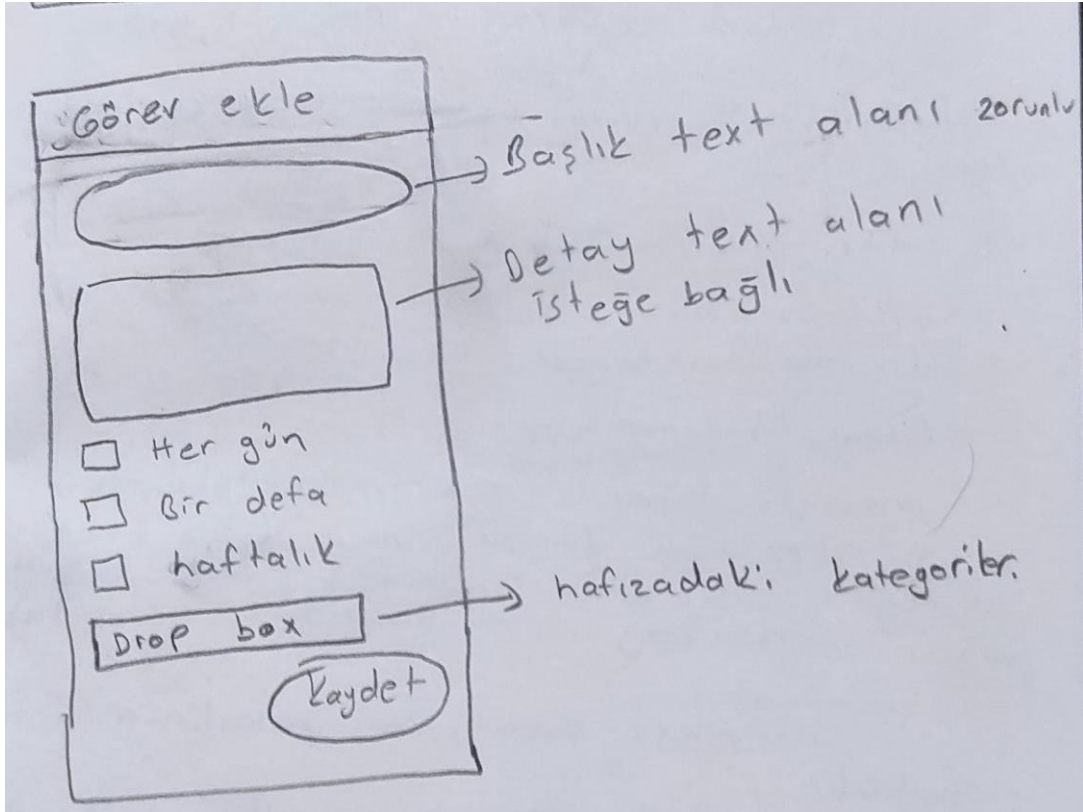
Şekil4.1 Anasayfa ve Tüm görevler Sayfası Tasarımı

Şekil 4.2.'de projeye ait kategori ekran tasarımı verilmiştir.



Şekil 4.2 Kategori ekran Tasarımı

4.3.'de projeye ait Görev Ekle ekran tasarımı verilmiştir.



Şekil 4.3 Görev Ekle ekran Tasarımı

4.2 Veritabanı

Bu projede, **SQLite** eklentisi kullanılarak SQLite tabanlı bir veritabanı tasarlanmıştır. Veritabanı, uygulamanın kategoriler ve görevleri (tasks) yönetebilmesi için iki temel tablo içermektedir:[5]

1. **categories** Tablosu:

- **id:** Otomatik artan birincil anahtar (INTEGER).
- **name:** Kategori adı (TEXT).
- **icon:** Kategoriye ait simge bilgisi (TEXT).

2. **tasks** Tablosu:

- **id:** Otomatik artan birincil anahtar (INTEGER).
- **date:** Görevin tarihi (TEXT).
- **title:** Görevin başlığı (TEXT).
- **category_id:** İlgili kategorinin kimliği (INTEGER). Bu sütun, **categories** tablosundaki id sütununa bir yabancı anahtar (FOREIGN KEY) ile bağlanmıştır.

Bu yapı, görevlerin belirli kategorilerle ilişkilendirilmesini sağlar. Ayrıca, veri bütünlüğü ve düzenli veri yönetimi için ilişkisel bir veritabanı yapısı sunmaktadır.

Şekil 4.4.'de projeye ait DB Tablo tasarımı verilmiştir.

```
await db.execute('''
    CREATE TABLE categories(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT,
        icon TEXT
    )
''');
await db.execute('''
    CREATE TABLE tasks(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        date TEXT,
        title TEXT,
        category_id INTEGER,
        FOREIGN KEY (category_id) REFERENCES categories(id)
    )
''');
```

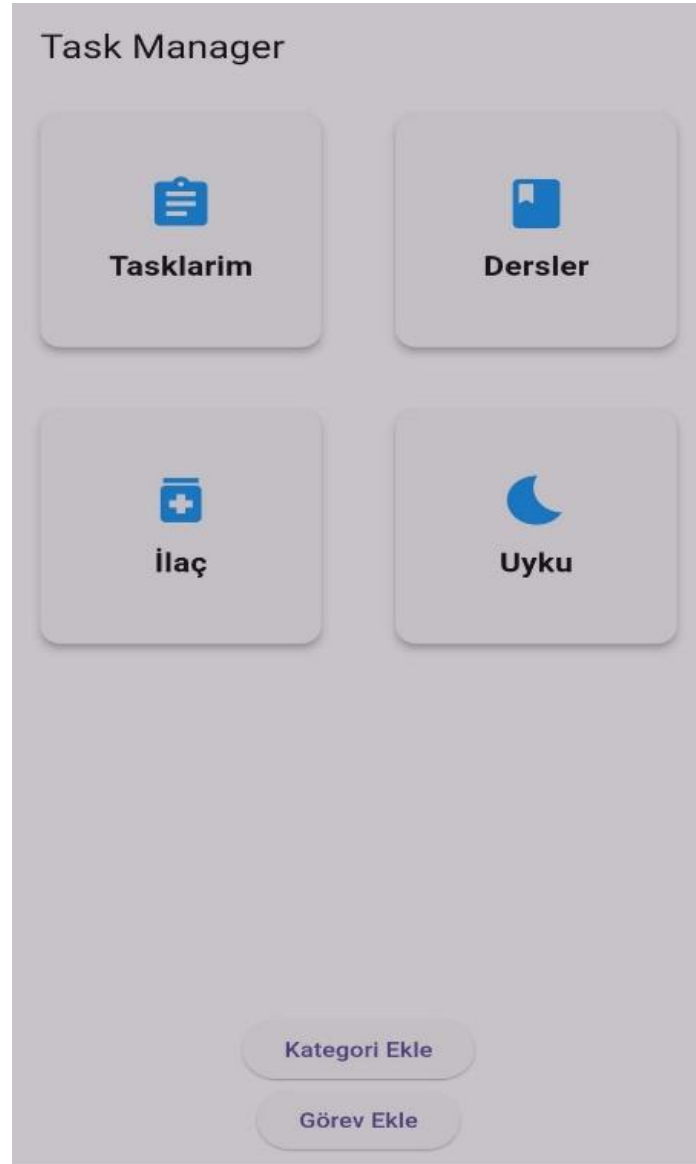
Şekil 4.4 DB Tablo tasarımı

4.3 Uygulama Arayüzleri

Uygulama, kullanıcı dostu bir arayüz ile tasarlanmıştır. Her bir ekran, sade bir tasarım ve işlevsellik göz önünde bulundurularak geliştirilmiştir. Kullanıcılar kategorileri ve görevlerini kolayca yönetebilmekte, yeni görevler ekleyebilmekte ve mevcut görevlerini düzenleyebilmektedir.

Bu bölümde yer alan tasarım ve veritabanı yapısı, uygulamanın temel işlevlerini sağlamanın yanı sıra kullanıcı deneyimini ön planda tutacak şekilde optimize edilmiştir.

Şekil 4.5.'de projeye ait Anasayfa Ekranı görüntüsü verilmiştir.



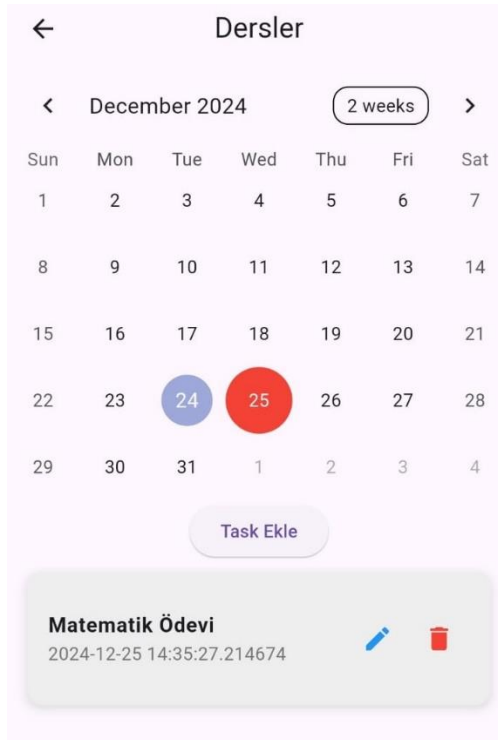
Şekil 4.5 Anasayfa Ekranı

Şekil 4.6.'de projeye ait Tüm Görevler Ekranı görüntüsü verilmiştir.



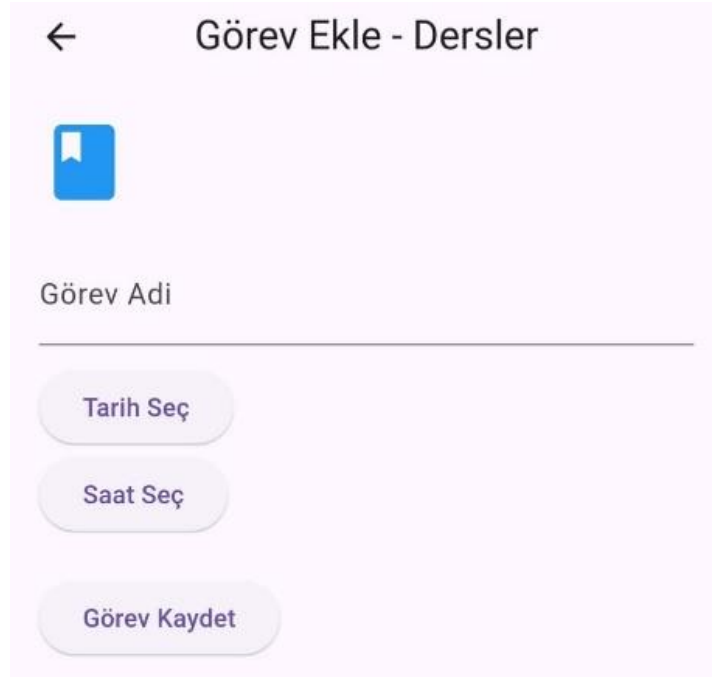
Şekil 4.6 Tüm Görevler Ekranı

Şekil 4.7.'de projeye ait Bir Kategoriye Ait Görevler Ekranı görüntüsü verilmiştir.



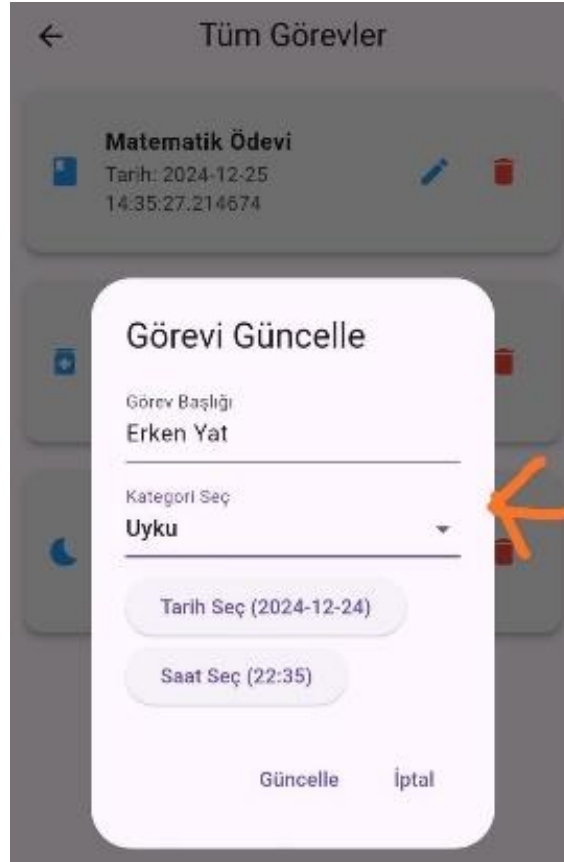
Şekil 4.7 Bir Kategoriye Ait Görevler Ekranı

Şekil 4.8.'de projeye ait Görev ekle Ekranı görüntüsü verilmiştir.



Şekil 4.8 Görev ekle Ekranı

Şekil 4.9.'de projeye ait Görev güncelle Ekranı görüntüsü verilmiştir.



Şekil 4.9 Görev güncelle Ekranı

5. SONUÇ VE TARTIŞMA

Klasik Takvime göre bazı yenilikçi ve özgün unsurlar içeren bir çözüm geliştirilmiştir. Ancak, bazı özellikler tamamlanamamış ve iyileştirilmesi gereken alanlar bulunmaktadır. Bu eksiklikler, ilerleyen süreçte tamamlanarak uygulamanın fonksiyonelliği artırılacaktır.

Uygulama Android üzerinde test edilmiştir. Web içinde yazılmıştır ama SQFLite Web üzerinde test edilememektedir. Gelecekte daha farklı bir ilişkisel veri tabanı kullanılarak iyileştirilebilir veya direk Firebase ile daha güzel bir backend ve veritabanı kullanılabilir.[8]

Uygulamayabildirimler (notification) kısmı eklenerek daha iyi bir hizmet verilebilir.

6. KAYNAKÇA

- [1] Table Calendar. [Offical Documantation](#)
- [2] [Google. \(2017\). Announcing Flutter: Building beautiful mobile apps with Dart. Resmi Flutter Blog Yazısı](#)
- [3] Google. (n.d.). *Android Studio Overview*. [Android Developers Resmi Web Sitesi](#)
- [4] Microsoft. (n.d.). *Visual Studio Code – Code Editing. Redefined*. [Visual Studio Code Resmi Web Sitesi](#)
- [5] Sqflite: *Flutter plugin for SQLite database management*. (n.d.). [Flutter Resmi Web Sitesi](#)
- [6] Sqflite Documentation: *Sqflite plugin documentation for Flutter*. (n.d.). [Resmi Dökümantasyon](#)
- [8] [Firebase Resmi Web Sitesi](#)

